



**Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)**

ОТЧЕТ ПО ПРАКТИКЕ
по основной образовательной программе
среднего профессионального образования по специальности
09.02.07 «Информационные системы и программирование»

Вид практики (учебная, производственная, преддипломная): _____

Установленный по КУГ срок прохождения практики: с __.__.20__г. по __.__.20__г.

Место прохождения практики (наименование организации):

Выполнил студент
__-го курса

(подпись)

(фамилия, имя, отчество)

Руководитель от
образовательной
организации

(подпись)

(ученая степень, фамилия, имя, отчество)

(должность)

Руководитель от
предприятия

(подпись)

(ученая степень, фамилия, имя, отчество)

(должность)

Оценка

(подпись)

Дата сдачи отчета: __.__.20__г.

Тверь, 2025 г.

Содержание

Введение.....	3
1. Реализация линейных структур данных	5
1.1 Массивы.....	5
1.2 Связные списки.....	5
2. Структуры управления потоком данных	7
2.1 Стеки и очереди.....	7
2.2 Применение: Калькулятор	7
3. Хэш-таблицы.....	8
3.1 Реализация	8
3.2 Практические применения.....	8
3.3 Автодополнение.....	9
4. Деревья и графы.....	10
4.1 Бинарное дерево поиска.....	10
4.2 Графы	10
4.3 Задача "Острова".....	11
5. Приоритетные структуры данных	12
5.1 Бинарная мин-куча.....	12
5.2 Приоритетная очередь	12
Заключение.....	14
Список источников.....	15
Приложение 1. Ссылка на репозиторий	16

Введение

Назначение и цели практики

Учебная практика УП.01 "Структуры данных" является фундаментальной дисциплиной в подготовке специалистов в области информационных технологий и программирования. Целью данной практики является углубленное изучение основных структур данных, их реализации и применения в решении прикладных задач.

Основные задачи практики

В процессе выполнения практических заданий решались следующие задачи:

1.

Освоение базовых структур данных - изучение и реализация массивов (статических и динамических), связанных списков (одно- и двусвязных), их операций и анализ временной сложности.

2.

Реализация структур управления потоком данных - разработка стеков и очередей на различных основаниях (массив, связный список, циклический массив), применение их для решения практических задач.

3.

Изучение хэш-таблиц и методов разрешения коллизий - создание собственной реализации хэш-таблицы, разработка хэш-функций, анализ производительности при различных сценариях.

4.

Освоение древовидных структур и графов - реализация бинарного дерева поиска, структуры Trie, представление и обход графов (BFS, DFS), решение задач на графах.

5.

Применение приоритетных структур - разработка бинарной кучи и приоритетной очереди, их использование в алгоритмах планирования и поиска.

Актуальность практики

Глубокое понимание структур данных является критически важным для:

- **Оптимизации производительности** приложений и алгоритмов
- **Правильного выбора** структуры данных для конкретной задачи
- **Анализа временной и пространственной сложности** решений
- **Разработки масштабируемых** систем и приложений
- **Подготовки к собеседованиям** на позиции разработчика

1. Реализация линейных структур данных

1.1 Массивы

В первом разделе практики были реализованы статический и динамический массивы с основными операциями.

Статический массив имеет фиксированный размер, задаваемый при создании. Операция `pushBack` выполняется за $O(1)$ - достаточно записать элемент в позицию `size` и увеличить счетчик. Операция `find` требует линейного поиска с сложностью $O(n)$ в худшем случае. Операции `pushFront`, `insert` и `remove` имеют сложность $O(n)$, так как требуют сдвига всех последующих элементов. Основной недостаток статического массива -

невозможность изменения размера после инициализации, что приводит к переполнению при превышении емкости.

Динамический массив решает проблему фиксированного размера через стратегию геометрического расширения. При заполнении массива создается новый массив удвоенного размера, в который копируются все существующие элементы. Хотя отдельные операции расширения требуют $O(n)$ времени, амортизированный анализ показывает среднюю стоимость $O(1)$ для последовательности операций `pushBack`. Экспериментальное тестирование с 100 000 элементами подтвердило эффективность подхода - время выполнения составило менее 0.1 секунды. Динамический массив сочетает преимущества быстрого доступа по индексу с гибкостью управления памятью.

1.2 Связные списки

Односвязный список состоит из узлов, каждый из которых содержит данные и ссылку на следующий элемент. Операция `pushFront` имеет сложность $O(1)$ - создается новый узел, который становится головой списка. Операция `pushBack` требует $O(n)$ в реализации без хвостового указателя, так как необходимо пройти весь список до последнего элемента. Реализована операция разворота списка на месте с использованием трех указателей (`prev`, `current`, `next`), которая выполняется

за $O(n)$ с пространственной сложностью $O(1)$. Сравнение с массивом показало преимущество списка при частых вставках в начало и недостаток при произвольном доступе.

Двусвязный список расширяет функциональность за счет хранения ссылок на предыдущий и следующий узлы. Это позволяет выполнять операцию `insertAfter` за $O(1)$ - новый узел вставляется между существующими с обновлением четырех ссылок. Операция `removeNode` также выполняется за $O(1)$ при наличии прямой ссылки на удаляемый узел, что исключает необходимость предварительного поиска. Реализован итератор, соответствующий протоколу Python, что позволяет использовать список в циклах `for`. Двусвязный список обеспечивает эффективные вставки и удаления в произвольных позициях, но требует дополнительной памяти для хранения обратных ссылок.

2. Структуры управления потоком данных

2.1 Стеки и очереди

Стек реализован в двух вариантах: на основе массива фиксированного размера и на основе связного списка. В массиве используется указатель `top`, операции `push` и `pop` выполняются за $O(1)$ через инкремент/декремент указателя и запись/чтение элемента. В списковом варианте новый элемент становится головой списка, что также обеспечивает $O(1)$. Классическое применение стека - проверка сбалансированности скобочных последовательностей. Алгоритм проходит по строке, помещая открывающие скобки в стек и проверяя соответствие закрывающих скобок вершине стека, что требует $O(n)$ времени и $O(n)$ памяти в худшем случае.

Очередь на циклическом массиве использует два указателя (`front` и `rear`) и массив фиксированного размера. При достижении конца массива указатели "заворачиваются" в начало, что позволяет эффективно использовать память. Операции `enqueue` и `dequeue` выполняются за $O(1)$ через обновление указателей и запись/чтение элементов. Очередь на двух стеках реализует функциональность через основной и вспомогательный стеки. При операции `dequeue`, если вспомогательный стек пуст, все элементы из основного стека перемещаются во вспомогательный. Амортизированная сложность операций составляет $O(1)$, хотя отдельные операции `dequeue` могут требовать $O(n)$ при перемещении элементов.

2.2 Применение: Калькулятор

Реализован алгоритм "Шунтирующая станция" для преобразования выражений из инфиксной нотации в обратную польскую нотацию (ОПН). Алгоритм обрабатывает входную строку, разделяя операнды и операторы. Операнды добавляются непосредственно в выходную очередь, операторы помещаются в стек с учетом приоритета: сложение и вычитание - 1, умножение и деление - 2, возведение в степень - 3. Скобки обрабатываются специально: открывающие помещаются в стек, закрывающие вызывают выталкивание операторов до открывающей скобки. Преобразование выполняется за $O(n)$, где n - длина выражения.

Вычисление выражения в ОПН использует стек для хранения промежуточных результатов. Алгоритм проходит по очереди ОПН: операнды помещаются в стек, операторы извлекают два верхних операнда, применяют операцию и помещают результат обратно в стек. Вычисление также выполняется за $O(n)$. Реализация поддерживает унарный минус, десятичные числа и проверку деления на ноль. Калькулятор демонстрирует практическое применение стека для обработки выражений со сложной структурой приоритетов.

3. Хэш-таблицы

3.1 Реализация

Реализована хэш-таблица с разрешением коллизий методом цепочек. Хэш-функция для строк использует полиномиальное хеширование: $\text{hash} = (\text{hash} * 31 + \text{ord}(\text{char})) \% \text{capacity}$, где 31 - простое число, обеспечивающее хорошее распределение. Таблица поддерживает коэффициент загрузки 0.75 - при его превышении выполняется рехеширование с удвоением емкости и перераспределением всех элементов.

Операция put вычисляет хэш-код ключа, находит соответствующий бакет и выполняет линейный поиск в цепочке. Если ключ найден, обновляется значение, иначе добавляется новый узел. Операция get аналогично ищет ключ в цепочке, возвращая значение или выбрасывая исключение. Операция remove удаляет узел из цепочки с обновлением ссылок соседних узлов. Средняя сложность операций составляет $O(1)$ при равномерном распределении ключей, в худшем случае (все ключи в одном бакете) деградирует до $O(n)$.

3.2 Практические применения

Частотный словарь строится за один проход по тексту: текст разбивается на слова с помощью регулярных выражений, слова приводятся к нижнему регистру, частота каждого слова увеличивается в хэш-таблице. Сложность построения - $O(n)$, где n - количество слов. Извлечение топ-10 самых частых слов требует сортировки словаря по значениям с сложностью $O(m \log m)$, где m - количество уникальных слов.

Сравнение хэш-функций показало значительную разницу в производительности. Плохая хэш-функция, всегда возвращающая 1, превращает хэш-таблицу в связный список с операциями $O(n)$. Хорошая хэш-функция обеспечивает равномерное распределение и операции $O(1)$. Эксперименты с текстом объемом 10 000 слов показали ускорение в 100-200 раз при использовании качественной хэш-функции.

3.3 Автодополнение

Реализована структура Trie (префиксное дерево) в комбинации с HashMap для хранения частот слов. Каждый узел Trie содержит словарь дочерних узлов, флаг завершения слова и частоту использования. При вставке слова создается цепочка узлов по символам слова, в конечном узле устанавливается флаг завершения и увеличивается частота.

Функция autocomplete принимает префикс, проходит по дереву до узла, соответствующего префиксу, затем рекурсивно собирает все слова в поддереве. Слова сортируются по убыванию частоты с использованием HashMap, хранящего частоты всех слов. Поиск по префиксу выполняется за $O(m)$, где m -- длина префикса, сбор и сортировка результатов - $O(k \log k)$, где k - количество найденных слов. Система эффективно работает для больших словарей и обеспечивает релевантные подсказки на основе частоты использования.

4. Деревья и графы

4.1 Бинарное дерево поиска

Реализовано бинарное дерево поиска с поддержкой балансировки. Каждый узел содержит значение, ссылки на левое и правое поддеревья, и высоту поддерева. Операция `insert` рекурсивно спускается по дереву, сравнивая значения, и вставляет новый узел в соответствующую позицию. После вставки выполняется обновление высот и проверка балансировки с применением вращений при необходимости.

Операция `search` аналогично спускается по дереву, возвращая `true/false`. Операция `delete` обрабатывает три случая: узел без потомков (просто удаляется), узел с одним потомком (заменяется потомком), узел с двумя потомками (заменяется минимальным элементом правого поддерева). После удаления также выполняется балансировка.

Реализованы три типа обхода: `in-order` (левый-узел-правый) дает отсортированную последовательность, `pre-order` (узел-левый-правый) используется для копирования структуры, `post-order` (левый-правый-узел) - для удаления дерева. Каждый обход выполняется за $O(n)$. Проверка сбалансированности вычисляет разницу высот поддеревьев для каждого узла, что требует $O(n)$ времени.

4.2 Графы

Граф представлен двумя способами: матрицей смежности и списком смежности. Матрица смежности - двумерный массив размера $V \times V$, где элемент $[i][j]$ указывает наличие ребра между вершинами i и j . Это представление эффективно для плотных графов, проверка ребра выполняется за $O(1)$, но требует $O(V^2)$ памяти.

Список смежности - массив списков, где каждый элемент содержит смежные вершины. Это представление эффективно для разреженных графов, требует $O(V+E)$ памяти. Реализованы алгоритмы BFS и DFS. BFS использует очередь, посещает вершины по уровням, находит кратчайшие пути в невзвешенном графе за $O(V+E)$. DFS использует стек (рекурсию), применяется для топологической сортировки, поиска компонент связности, также за $O(V+E)$.

Алгоритм поиска кратчайшего пути в невзвешенном графе использует BFS с сохранением предшественников. После завершения обхода путь восстанавливается от конечной вершины к начальной через цепочку предшественников. Сложность - $O(V+E)$.

4.3 Задача "Острова"

Реализовано решение задачи подсчета островов в бинарной матрице. Остров — компонента связности единиц, связанных по горизонтали, вертикали и диагонали. Алгоритм использует DFS для обхода каждого острова.

Инициализируется матрица посещенных ячеек того же размера, что и входная матрица. Алгоритм проходит по всем ячейкам. При обнаружении не посещённой единицы увеличивается счетчик островов и запускается DFS. DFS помечает все связанные единицы как посещенные, используя стек или рекурсию. Обход учитывает 8 направлений: север, юг, запад, восток и диагонали.

Сложность алгоритма - $O(\text{rows} \times \text{cols})$, так как каждая ячейка посещается один раз. Пространственная сложность - $O(\text{rows} \times \text{cols})$ для матрицы посещений в реализации без модификации исходной матрицы. Дополнительно реализована функция измерения размеров островов, которая возвращает список площадей всех островов.

5. Приоритетные структуры данных

5.1 Бинарная мин-куча

Реализована бинарная мин-куча на основе массива. Куча поддерживает свойство: значение каждого узла не превышает значений его потомков. Для узла с индексом i потомки имеют индексы $2i+1$ и $2i+2$, родитель - $(i-1)//2$.

Операция `insert` добавляет элемент в конец массива, затем выполняет "всплытие": сравнивает элемент с родителем и меняет местами при нарушении свойства кучи. Сложность - $O(\log n)$. Операция `extractMin` извлекает корневой элемент (минимум), заменяет его последним элементом массива, затем выполняет "погружение": сравнивает элемент с потомками и меняет с минимальным при нарушении свойства. Сложность - $O(\log n)$.

Операция `buildHeap` преобразует произвольный массив в кучу за $O(n)$, а не $O(n \log n)$, путем применения операции погружения ко всем элементам от середины массива к началу. Проверка корректности кучи проходит все узлы, проверяя свойство кучи для каждого, что требует $O(n)$ времени.

5.2 Приоритетная очередь

На основе мин-кучи реализована приоритетная очередь. Каждый элемент представляет собой пару (значение, приоритет). Операция `push` добавляет элемент в кучу с учетом приоритета, операция `pop` извлекает элемент с минимальным приоритетом. Обе операции имеют сложность $O(\log n)$.

Применение к планированию задач: задачи с разными приоритетами помещаются в очередь, извлекаются в порядке приоритета. Реализован симулятор, который обрабатывает задачи за единицы времени, ведя учет времени начала и завершения.

Применение к поиску k минимальных элементов: все элементы массива помещаются в приоритетную очередь, затем извлекаются k элементов. Сложность — $O(n \log k)$, что эффективнее полной сортировки $O(n \log n)$ при $k \ll n$. Альтернативный алгоритм использует `max`-кучу размера k для хранения k минимальных элементов при однократном проходе по массиву.

Дополнительно реализована операция слияния отсортированных списков: в очередь помещаются первые элементы каждого списка, на каждом шаге извлекается минимальный элемент и в очередь добавляется следующий элемент из того же списка. Сложность — $O(N \log k)$, где N — общее количество элементов, k — количество списков.

.

Заключение

В ходе практики успешно реализованы и протестированы все основные структуры данных. Линейные структуры обеспечивают различные компромиссы между скоростью доступа и модификации. Стеки и очереди оптимизированы для специфических паттернов доступа. Хэш-таблицы обеспечивают быстрый поиск, качество которого зависит от хэш-функции. Деревья и графы позволяют эффективно представлять иерархические и сетевые структуры. Приоритетные структуры критичны для алгоритмов оптимизации.

Полученные знания необходимы для разработки производительных приложений и выбора оптимальных структур данных для конкретных задач.

Список источников

1. Кормен, Т. Алгоритмы: построение и анализ / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. – 3-е изд. – М. : Вильямс, 2013. – 1328 с.
2. Скиена, С. Алгоритмы. Руководство по разработке / С. Скиена. – 2-е изд. – СПб. : БХВ-Петербург, 2014. – 720 с.
3. Макконнелл, Дж. Анализ алгоритмов. Активный обучающий подход / Дж. Макконнелл. – М. : Техносфера, 2009. – 416 с.
4. Python Software Foundation. The Python Standard Library [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/index.html> (дата обращения: 11.12.2025).
5. Python Software Foundation. Data Structures [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/tutorial/datastructures.html> (дата обращения: 11.12.2025).

Приложение 1. Ссылка на репозиторий



Ссылка для ручного ввода

<https://github.com/Hioka3/praktika>



**Частное учреждение профессионального образования
«Высшая школа предпринимательства»
(ЧУПО «ВШП»)**

ДНЕВНИК ПРАКТИКИ

Вид практики (учебная, производственная, преддипломная): _____

Установленный по КУГ срок прохождения практики: с _____.20__г. по _____.20__г.

Место прохождения практики (наименование организации):

Выполнил студент
__-го курса

(подпись)

(фамилия, имя, отчество)

Руководитель от
образовательной
организации

(подпись)

(ученая степень, фамилия, имя, отчество)

(должность)

Руководитель от
предприятия

(подпись)

(ученая степень, фамилия, имя, отчество)

(должность)

Тверь, 2025 г.

Дата	Краткое содержание, описывающее активность студента на указанную дату	Комментарий руководителя	Оценка руководителя	Подпись руководителя

Руководитель практики:

_____ / _____

(подпись)

(Ф.И.О.)

Тверь, 2025 г.